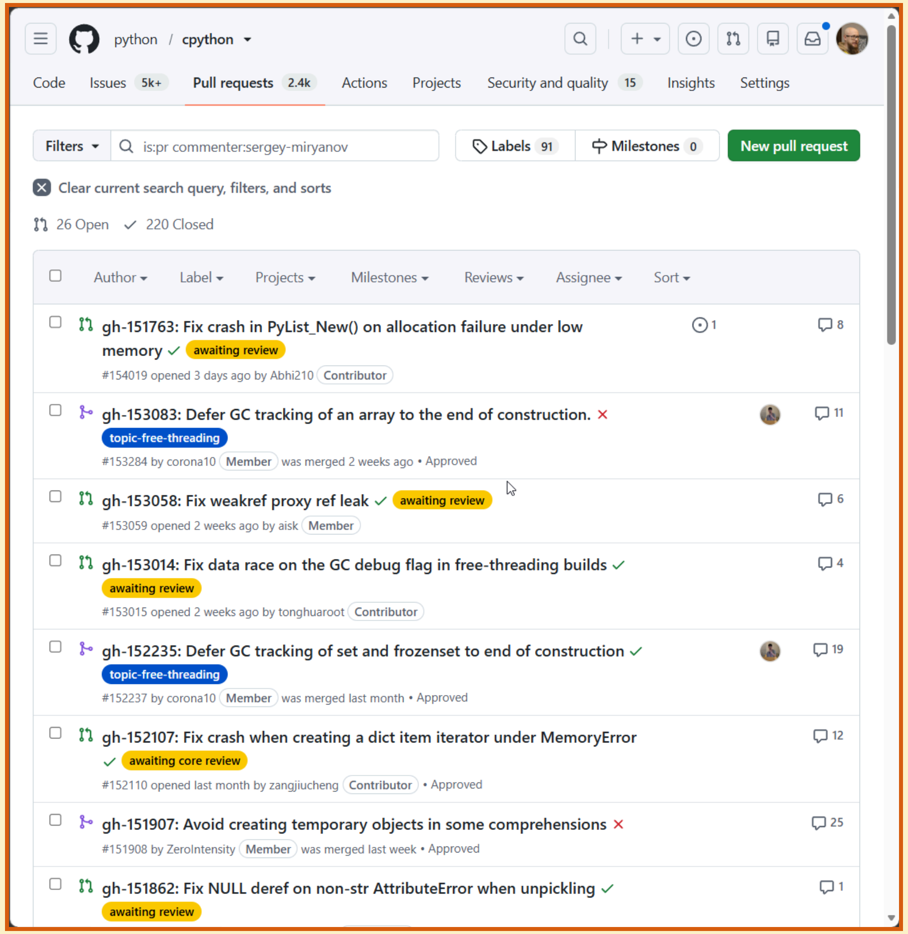
Мониторинг событий сборщика мусора в Python

Вчера, сегодня, завтра?

Мирянов Сергей

PyCon Russia 2026

CPYTHON КОНТРИБЬЮТОР



О ЧЁМ БУДЕМ ГОВОРИТЬ

- Управление памятью
- Сборка мусора (GC, Garbage Collector, Garbage Collection)
- События сборки мусора

ПРОБЛЕМА

Проблема

Инструменты

Новое API

gcmon

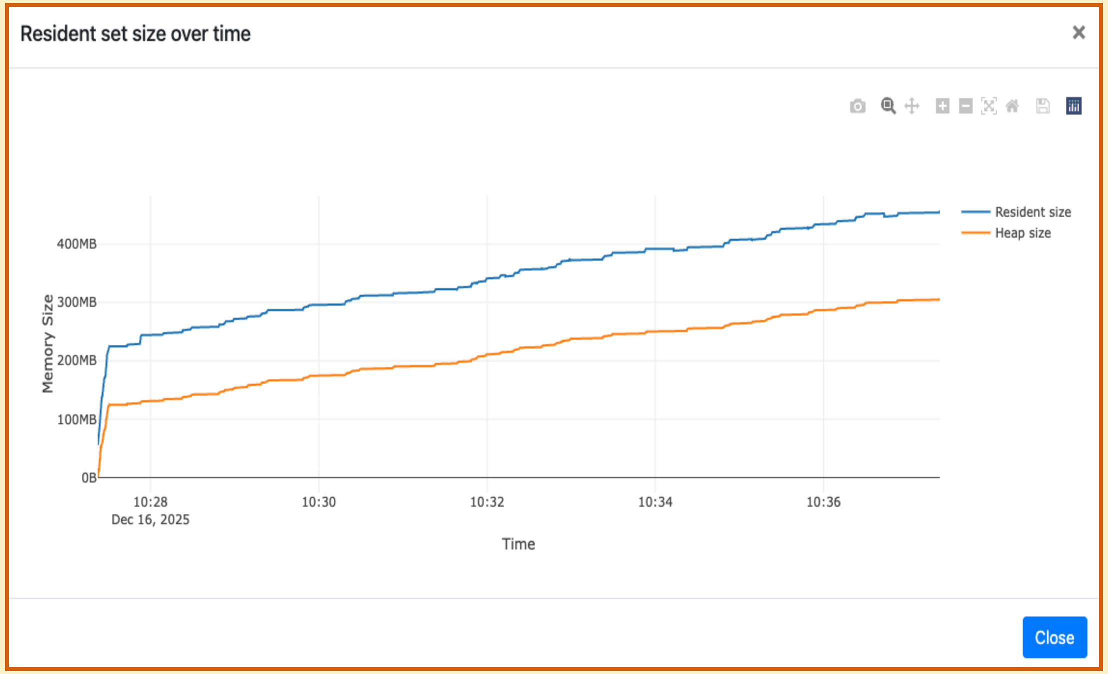
Perfetto

Бенчмарки

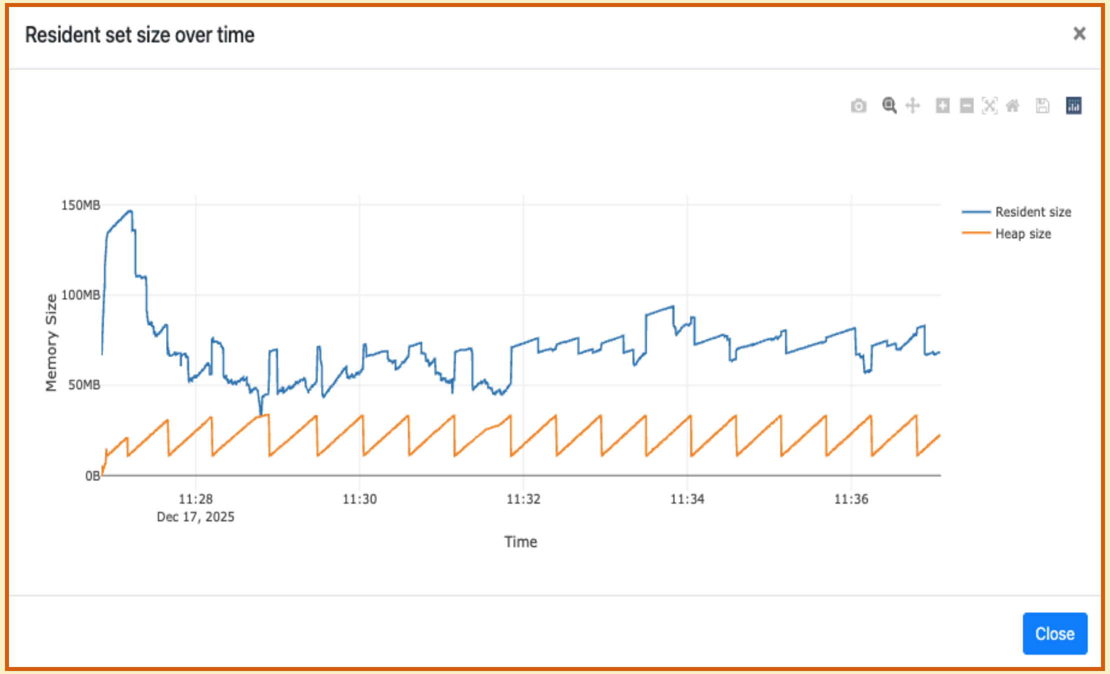
Backports

https://github.com/python/cpython/issues/142516

3.14



3.13

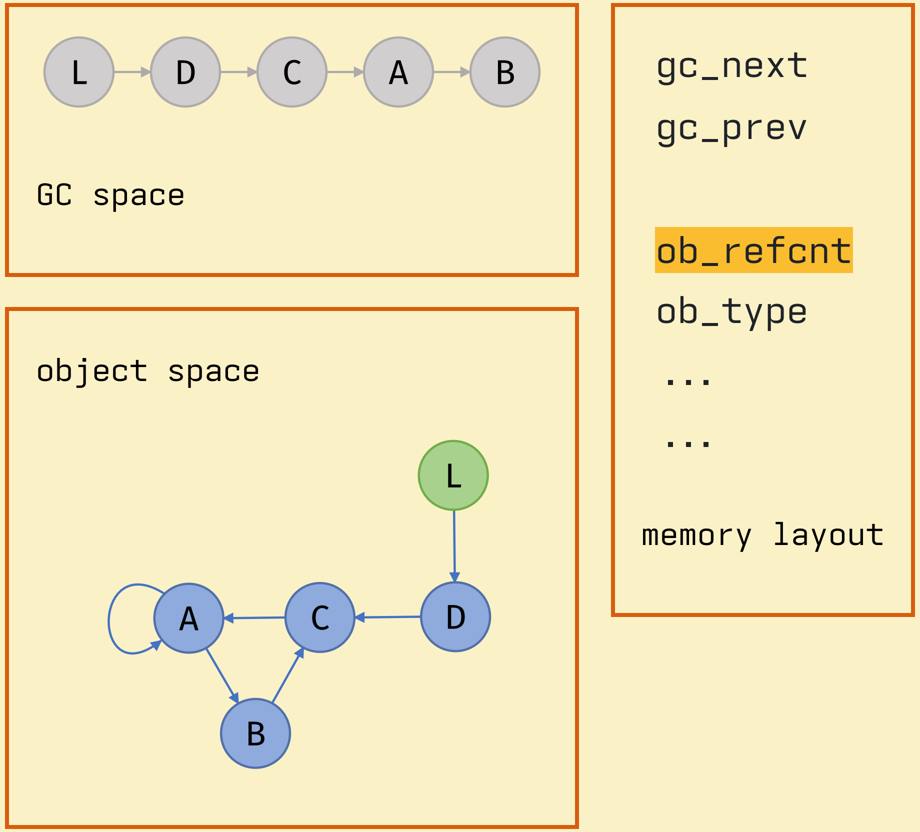


ПОЧЕМУ ПОТРЕБЛЕНИЕ ПАМЯТИ РАСТЁТ?

- Естественное состояние?
- Внутренняя утечка?
- Внешняя утечка?
- Ошибка в Python runtime/stdlib?
- Ошибка в GC?

ЧТО ТАКОЕ УТЕЧКА ПАМЯТИ?

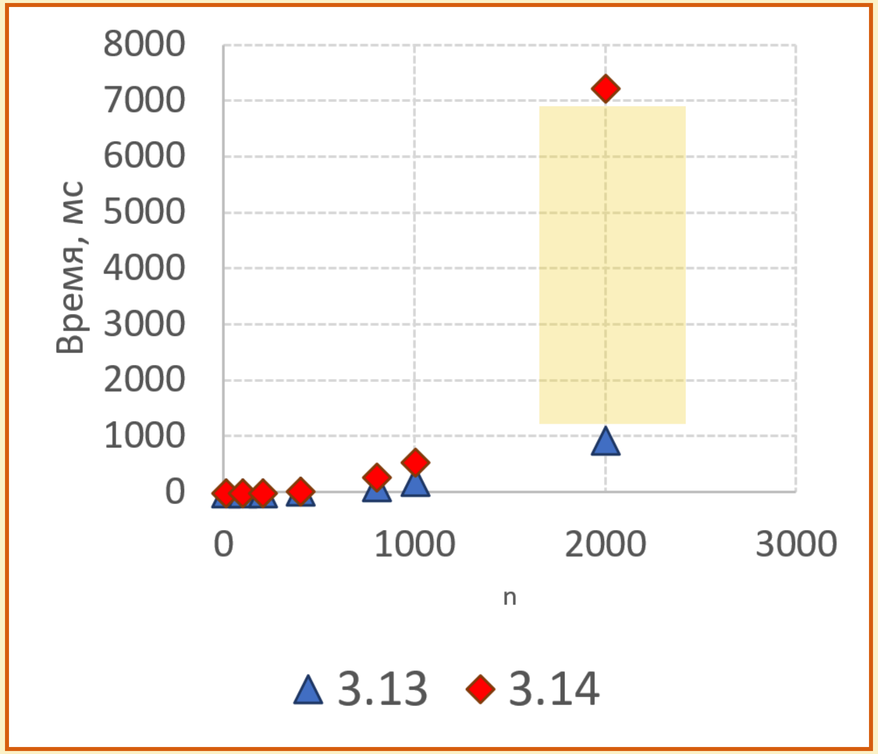
- Забыли обновить счётчик ссылок
- Не освободили неуправляемый блок памяти
- Продлили время жизни
- Сборщик мусора не разорвал циклы



ЗА ЧТО ОТВЕЧАЕТ GC?

- Определяет недостижимые объекты
- Вызывает финализаторы
- Обрабатывает слабые ссылки
- Разрывает циклы
- *Удаляет объекты*

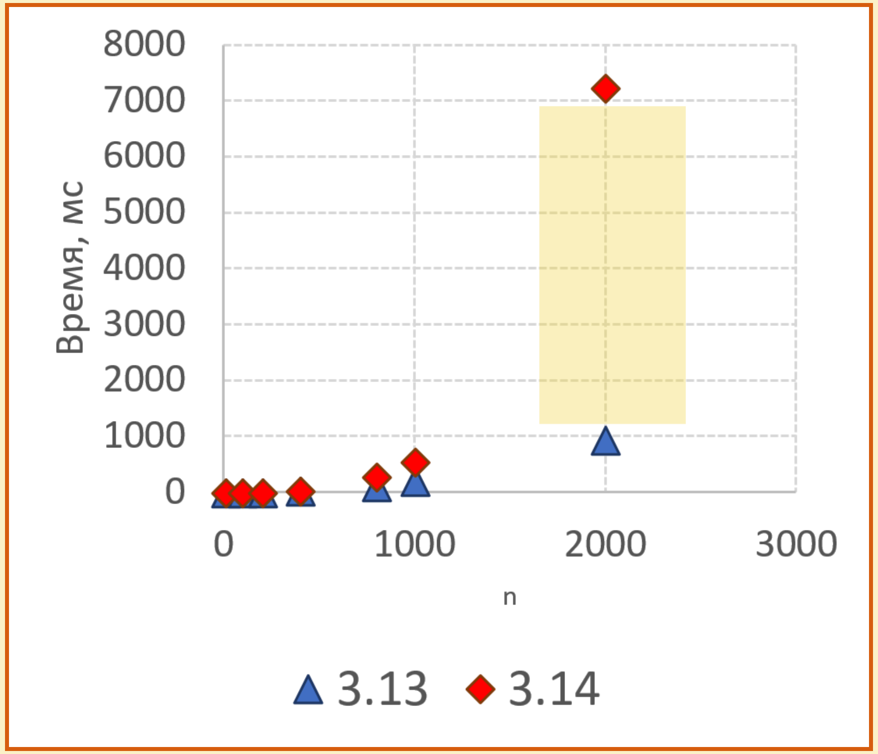
```python
def test(n=2000):
    d = {}
    for i in range(n):
        for j in range(n):
            d[(i,j)] = i
```



ЗА ЧТО ОТВЕЧАЕТ GC?

- **3.14.0-3.14.4**
  - **Определяет живые объекты**
  - **Формирует инкремент**
- Определяет недостижимые объекты
- Вызывает финализаторы
- Обрабатывает слабые ссылки
- Разрывает циклы
- *Удаляет объекты*

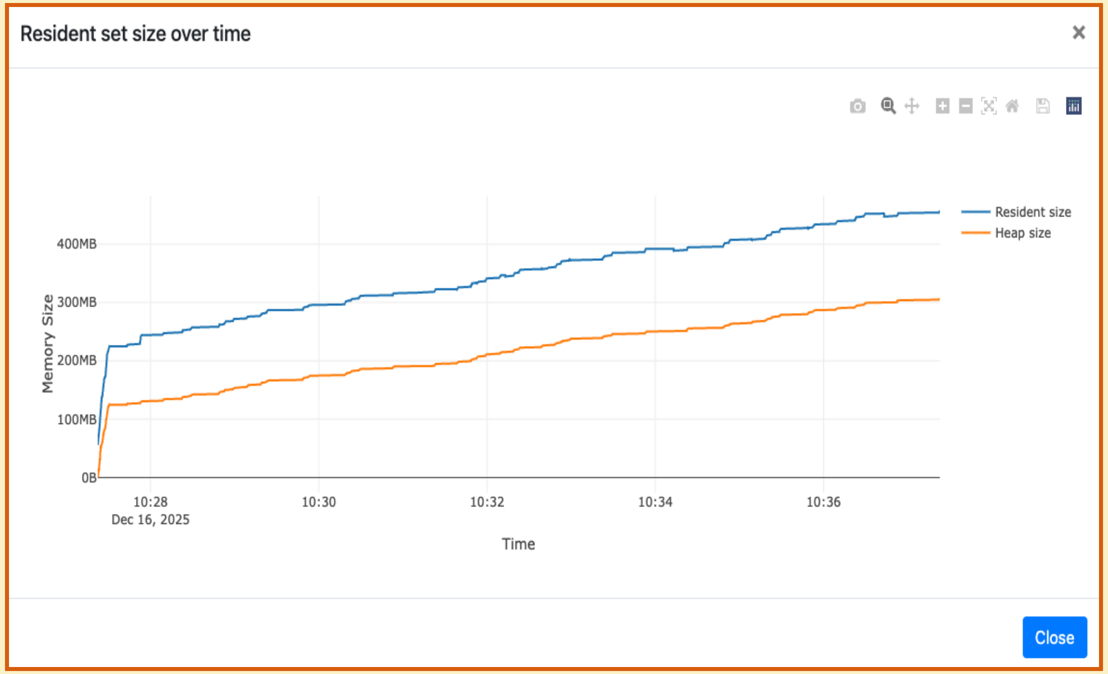
```python
def test(n=2000):
    d = {}
    for i in range(n):
        for j in range(n):
            d[(i,j)] = i
```



Подробнее здесь →

https://github.com/python/cpython/issues/142516

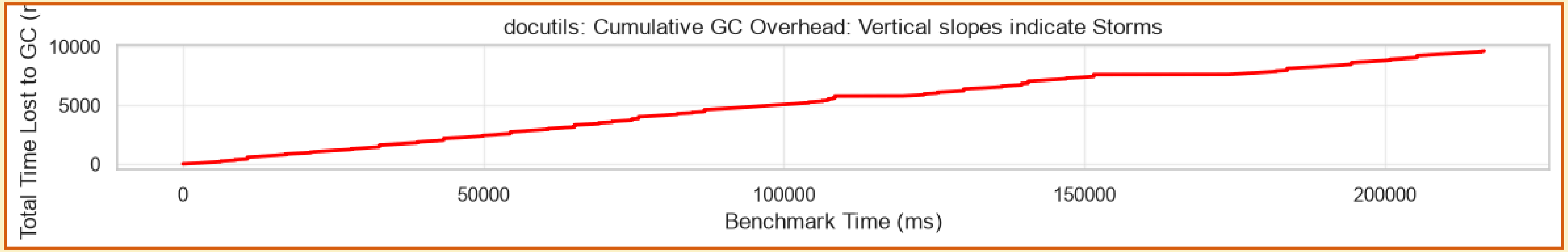
3.14




```
gc: collectable <Response 0x7fd8e3c4cc20>
  → <bytes 0x561a014e9218 rc=3221225472> <b''>
  → <IdentityDecoder 0x7fd8e3c4d010 rc=1> «httpx._decoders.IdentityDecoder obje
  → <datetime.timedelta 0x7fd8e551a490 rc=1> <datetime.timedelta(microseconds=32
  → <int 0x561a014e71f8 rc=3221225472> <0>
  → <Request 0x7fd8e3cf4ec0 rc=1> «Request('HEAD', 'http://www.google.com/')»
  → <str 0x7fd8e52c9650 rc=17> <'utf-8'>
  → <dict 0x7fd8e3c2c540 rc=1> <{'http_version': b'HTTP/1.1', 'reason_phrase': b
  → <Headers 0x7fd8e3d6ae00 rc=1> <Headers([('content-type', 'text/html; charset
  → <list 0x7fd8e3c2c900 rc=1> <[]>
  → <bool 0x561a014a73c0 rc=3221225472> <True>
  → <bool 0x561a014a73c0 rc=3221225472> <True>
  → <NoneType 0x561a014be300 rc=3221225472> <None>
  → <int 0x561a014e8af8 rc=3221225472> <200>
  → <BoundAsyncStream 0x7fd8e3c4cec0 rc=1> «httpx._client.BoundAsyncStream obje
    → <Response 0x7fd8e3c4cc20 rc=1> «Response [200 OK]»
      → <bytes 0x561a014e9218 rc=3221225472> <b''>
      → <IdentityDecoder 0x7fd8e3c4d010 rc=1> «httpx._decoders.IdentityDecoder
      → <datetime.timedelta 0x7fd8e551a490 rc=1> <datetime.timedelta(microsecond
      → <int 0x561a014e71f8 rc=3221225472> <0>
      → <Request 0x7fd8e3cf4ec0 rc=1> «Request('HEAD', 'http://www.google.com/'
      → <str 0x7fd8e52c9650 rc=17> <'utf-8'>
      → <dict 0x7fd8e3c2c540 rc=1> <{'http_version': b'HTTP/1.1', 'reason_phrase
      → <Headers 0x7fd8e3d6ae00 rc=1> <Headers([('content-type', 'text/html; cha
      → <list 0x7fd8e3c2c900 rc=1> <[]>
      → <bool 0x561a014a73c0 rc=3221225472> <True>
      → <bool 0x561a014a73c0 rc=3221225472> <True>
      → <NoneType 0x561a014be300 rc=3221225472> <None>
      → <int 0x561a014e8af8 rc=3221225472> <200>
      → <BoundAsyncStream 0x7fd8e3c4cec0 rc=1> «httpx._client.BoundAsyncStream
      → <type 0x561a239058b0 rc=30> «class 'httpx.Response'»
    → <float 0x7fd8e3cdd3b0 rc=1> <39037.104077293>
    → <AsyncResponseStream 0x7fd8e3cf6120 rc=1> «httpx._transports.default.Asyn
    → <type 0x561a23924160 rc=15> «class 'httpx._client.BoundAsyncStream'»
  → <type 0x561a239058b0 rc=30> «class 'httpx.Response'»
```

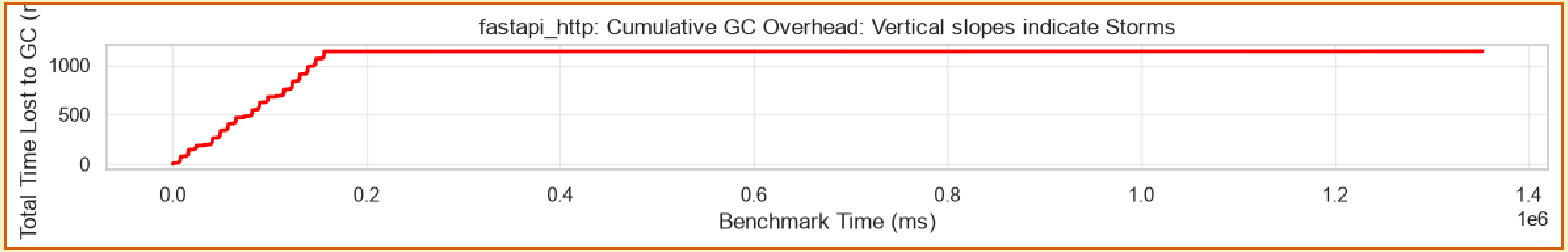
GC STORM

- Длинные/частые паузы
- Повышенное потребление памяти/CPU
- Частая полная сборка мусора



GC STORM

- Длинные/частые паузы
- Повышенное потребление памяти/CPU
- Частая полная сборка мусора



РЕЗЮМЕ

- Причины роста потребления памяти
- Что является утечкой
- Как работает подсчёт ссылок
- Как работает сборщик мусора
- Как влияет сборщик мусора на программу
- Что такое GC storm/thrashing

ИНСТРУМЕНТЫ

Проблема

Инструменты

Новое API

gcmon

Perfetto

Бенчмарки

Backports

ИНСТРУМЕНТЫ

- **In-process**
  - `tracemalloc`
  - `pympler`
  - APM (ddtrace, dynatrace, ...)
- **Out-of-process**
  - `memray`
  - `austin`, `tachyon`
  - `pyroscope`, `perforator`

- CPU
- Аллокации
- Снепшоты
- Циклы
- Время жизни
- Утечки
- Количество GC пауз
- Количество собранных объектов

МОДУЛЬ GC

- **Что не так с `gc.callbacks`**
  - Intrusive
  - C->Python interop
  - Сабинтерпретаторы
  - Heisenberg effect
- **Что не так с `gc.get_stats`**
  - Intrusive
  - When
  - Сабинтерпретаторы

НОВОЕ API

- Проблема
- Инструменты
- **Новое API**
- gcmon
- Perfetto
- Бенчмарки
- Backports

GCMONITOR

- _remote_debugging.get_gc_stats
- _remote_debugging.GCMonitor
  - init(pid)
  - get_gc_stats

- Внешний процесс
- Стандартные метрики
  - Collections
  - Collected
  - Uncollectable
  - *Candidates*
  - *Duration*
- *Количество живых объектов*
- Время начала и окончания паузы
- Поддержка сабинтерпретаторов

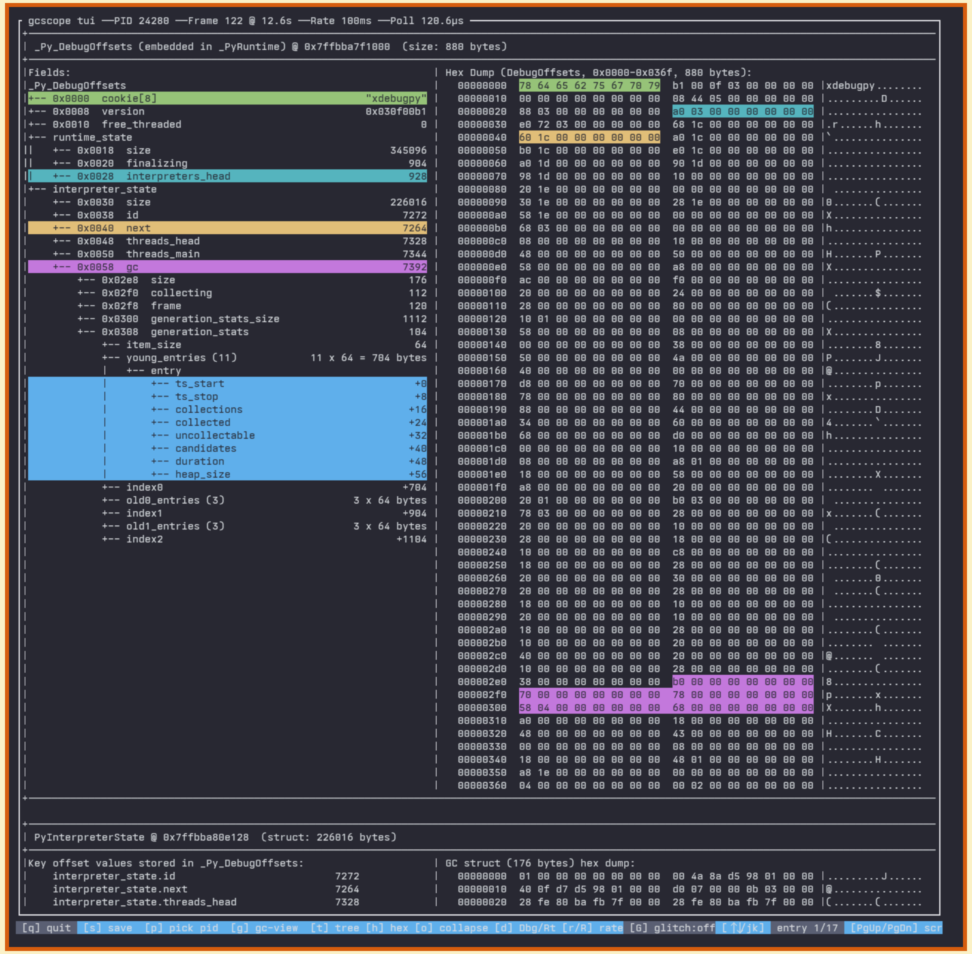
GCMONITOR

```c
struct pyruntimestate {
    /* This field must be first to facilitate locating it by out of process
     * debuggers. Out of process debuggers will use the offsets contained in
     * this field to be able to locate other fields in several interpreter
     * structures in a way that doesn't require them to know the exact layout
     * of those structures.
     *
     * IMPORTANT:
     * This struct is **NOT** backwards compatible between minor version of the
     * interpreter and the members, order of members and size can change
     * between minor versions. This struct is only guaranteed to be stable
     * between patch versions for a given minor version of the interpreter.
     */
    _Py_DebugOffsets debug_offsets;
    // ...
    struct pyinterpreters {
        PyMutex mutex;
        /* The linked list of interpreters, newest first. */
        PyInterpreterState *head;
        PyInterpreterState *main;
        int64_t next_id;
    } interpreters;
    // ...
};
```

```c
typedef struct _Py_DebugOffsets {
    char cookie[8] _Py_NONSTRING;
    uint64_t version;
    uint64_t free_threaded;
    struct _runtime_state {
        uint64_t size;
        uint64_t finalizing;
        uint64_t interpreters_head;
    } runtime_state;
    struct _interpreter_state {
        uint64_t size;
        uint64_t id;
        uint64_t next;
        uint64_t gc;
        // ...
    } interpreter_state;
    struct _gc {
        uint64_t size;
        uint64_t collecting;
        uint64_t frame;
        uint64_t generation_stats_size;
        uint64_t generation_stats;
    } gc;
} _Py_DebugOffsets;
```

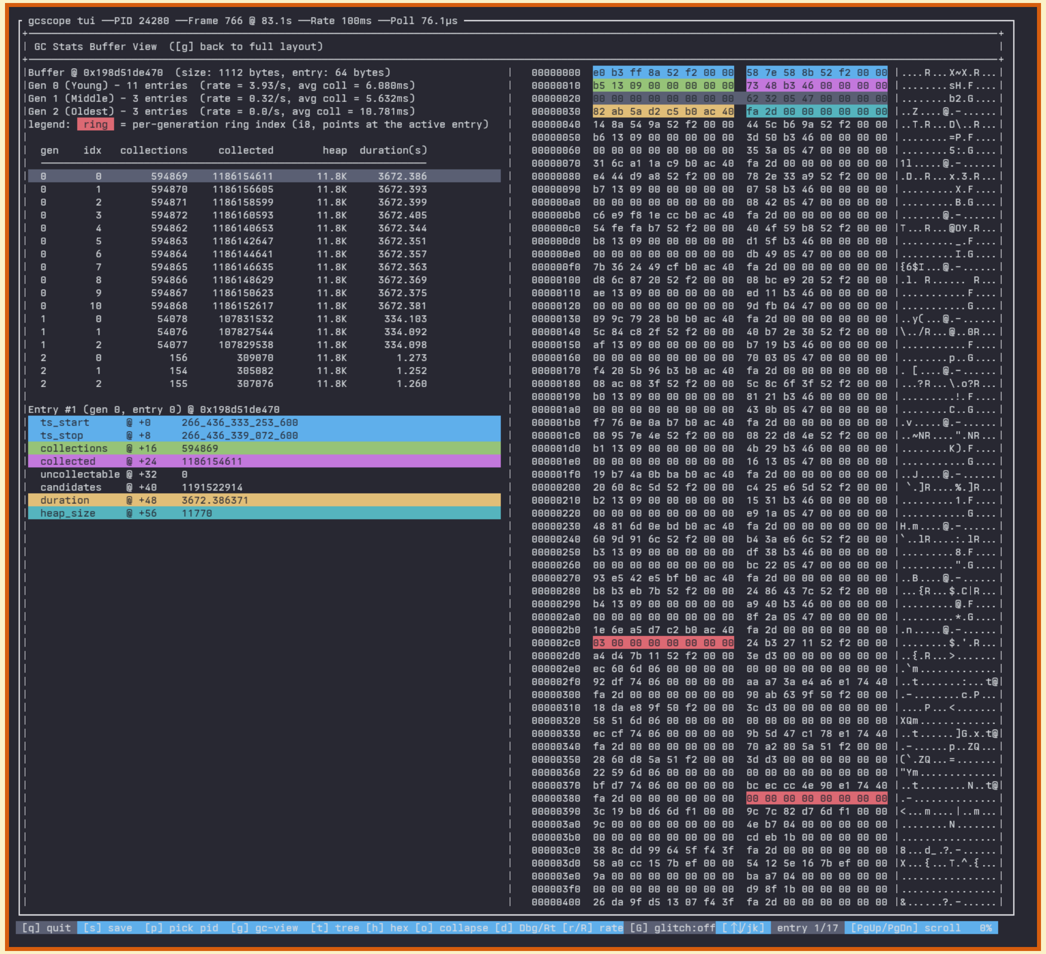
GCMONITOR

```c
typedef struct _Py_DebugOffsets {
    char cookie[8] _Py_NONSTRING;
    uint64_t version;
    uint64_t free_threaded;
    struct _runtime_state {
        uint64_t size;
        uint64_t finalizing;
        uint64_t interpreters_head;
    } runtime_state;
    struct _interpreter_state {
        uint64_t size;
        uint64_t id;
        uint64_t next;
        uint64_t gc;
        // ...
    } interpreter_state;
    struct _gc {
        uint64_t size;
        uint64_t collecting;
        uint64_t frame;
        uint64_t generation_stats_size;
        uint64_t generation_stats;
    } gc;
} _Py_DebugOffsets;
```



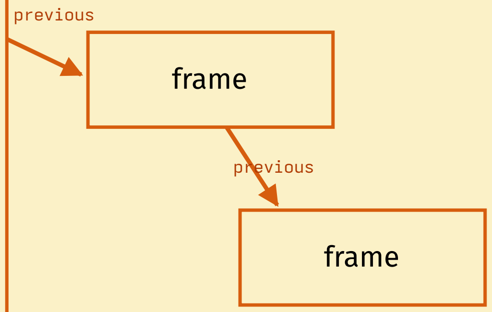
GC СТАТИСТИКА

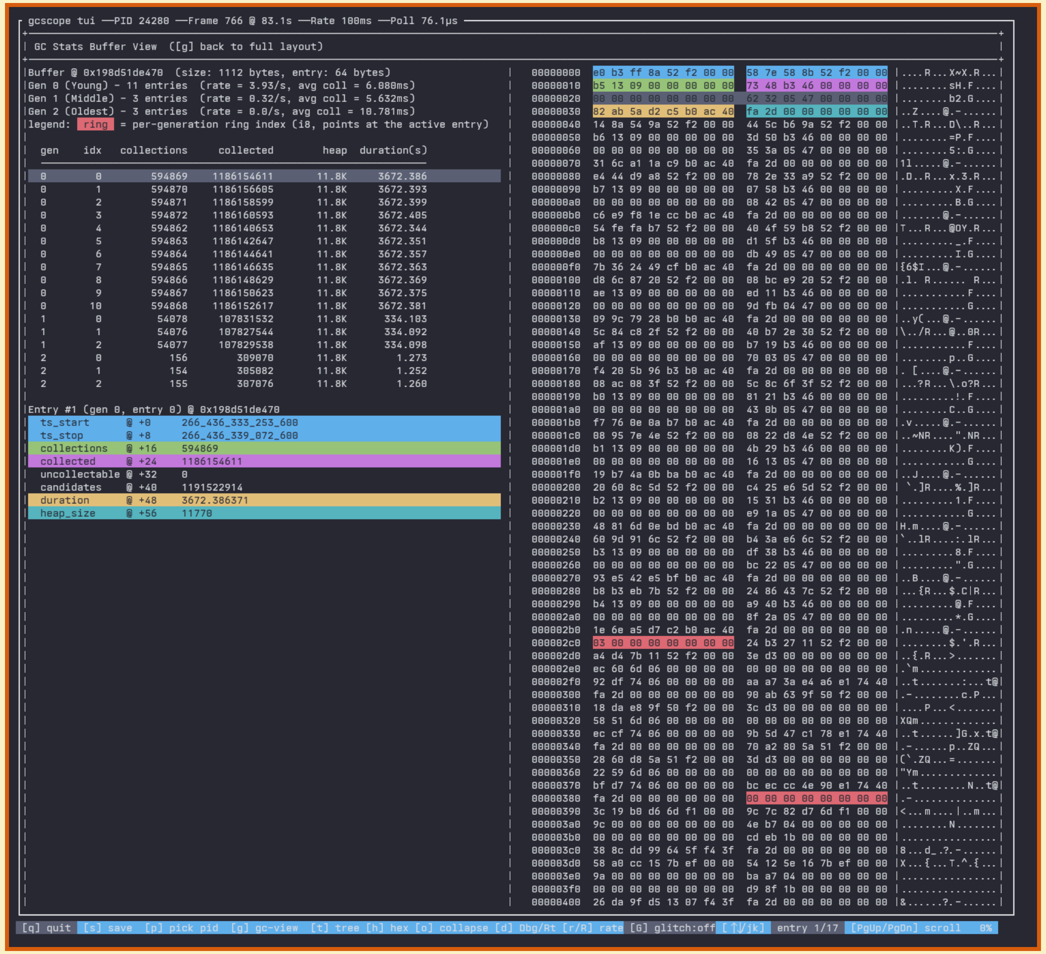
```c
struct gc_generation_stats {
    PyTime_t ts_start;
    PyTime_t ts_stop;
    Py_ssize_t collections;
    Py_ssize_t collected;
    Py_ssize_t uncollectable;
    Py_ssize_t candidates;
    double duration;
    Py_ssize_t heap_size;
};
struct gc_young_stats_buffer {
    struct gc_generation_stats items[11];
    int8_t index;
};
struct gc_old_stats_buffer {
    struct gc_generation_stats items[3];
    int8_t index;
};
struct gc_stats {
    struct gc_young_stats_buffer young;
    struct gc_old_stats_buffer old[2];
};
```



ПАУЗЫ

```
struct _PyInterpreterFrame {
_PyStackRef f_executable;
struct _PyInterpreterFrame *previous;
_PyStackRef f_funcobj;
PyObject *f_globals;
PyObject *f_builtins;
PyObject *f_locals;
PyFrameObject *frame_obj;
_Py_CODEUNIT *instr_ptr;
_PyStackRef *stackpointer;
uint16_t return_offset;
char owner;
uint8_t visited;
/* Locals and stack */
_PyStackRef localsplus[1];
};
```





ПОДВОДНЫЕ КАМНИ

- Не-питоновские процессы
- Дочерние процессы
- Состояние гонки
  - Не полная инициализация
  - Частичное чтение
- Повышенные права
- GC storm/thrashing

РЕЗЮМЕ

- Существующие инструменты отвечают на другие вопросы
- Использование модуля GC вносит искажения в замеры
- Новое API позволяет заглянуть внутрь GC и не вносит искажений
- Новое API не требует пауз
- Но есть подводные камни, которые надо учитывать

GCMON

Проблема
Инструменты
Новое API
gcmon
Perfetto
Бенчмарки
Backports

GCMON

- **Управление процессами**
  - Дочерние процессы
  - Время жизни
  - Метаданные
  - RSS
- **Расширенные метрики**
- **Экспорт**
  - Разные форматы
- Поддержка `pyperf`
- Метки

- **Оверхед**
  - Rate=100ms -> 1%
  - Rate=10ms -> 3-4%
- А что с gcscope?

- Perfetto Binary Format
- Chrome Trace Format
- JSONL/stdout
- OpenTelemetry?

GCMON

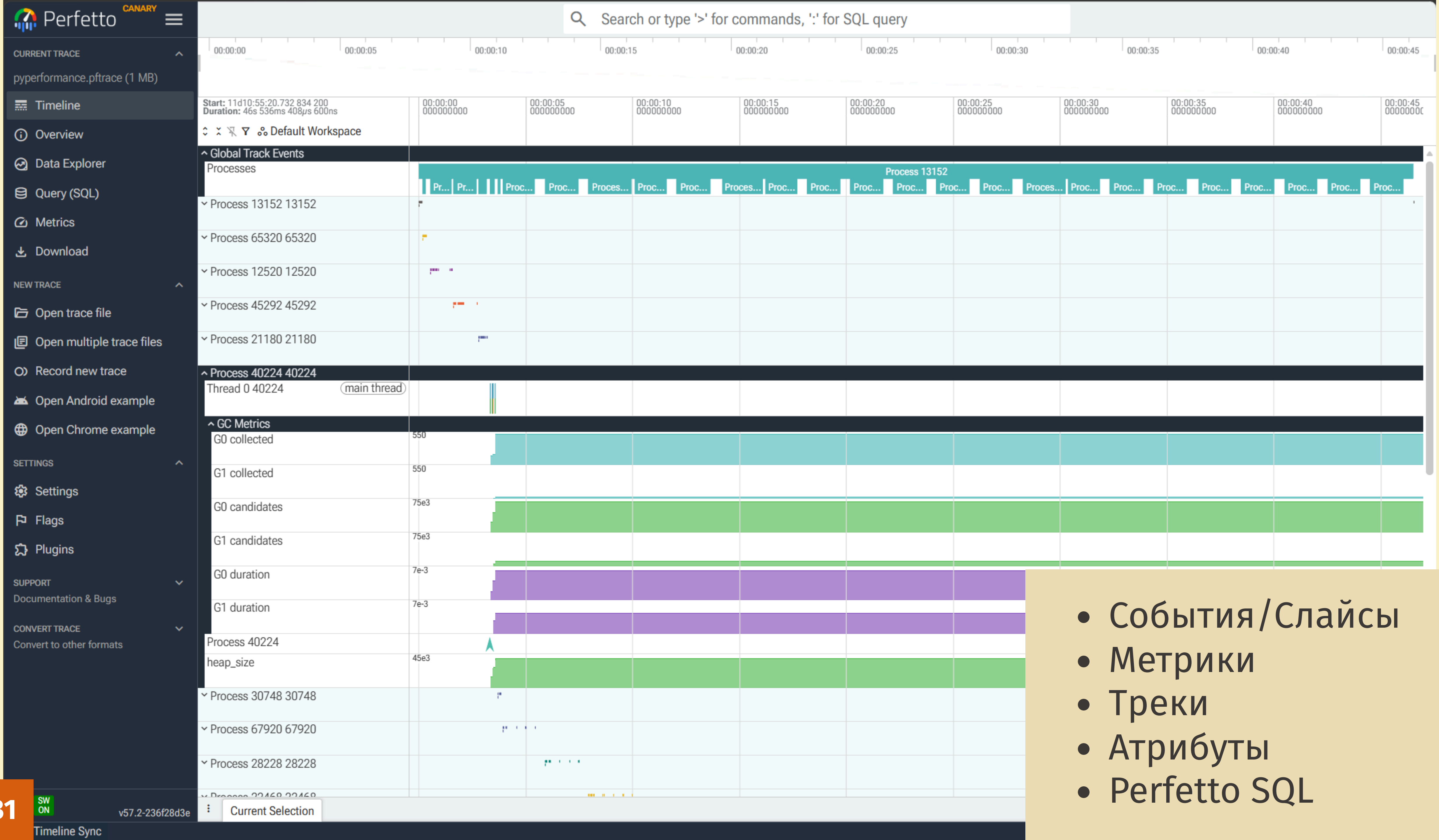
- **Стандартные метрики**
  - Collections
  - Collected
  - Uncollectable
  - *Candidates*
  - *Duration*
- *Количество живых объектов*
- Время начала и окончания паузы
- RSS

GCMON

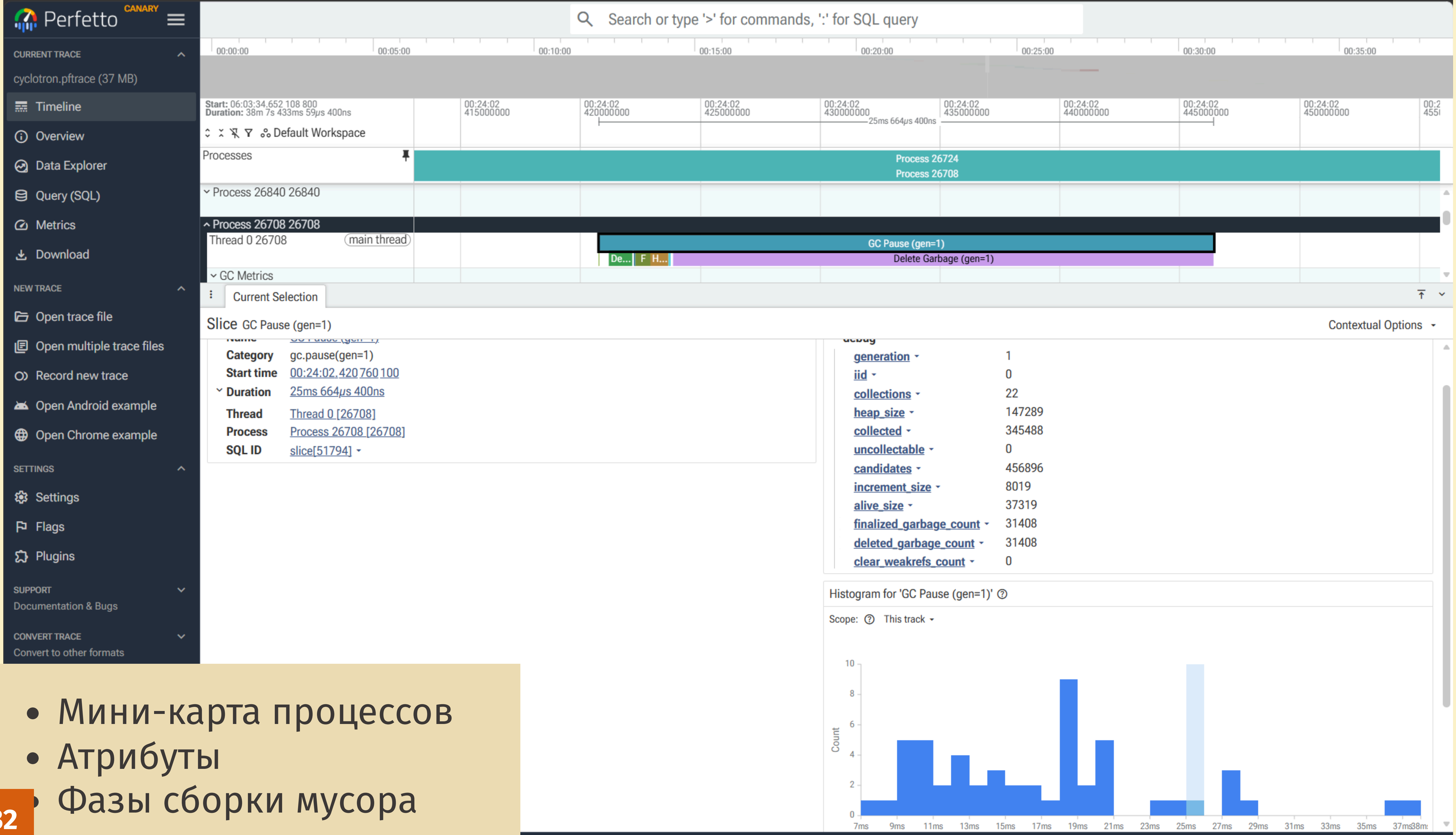
- **Отладочные данные**
  - Mark Alive
  - Fill Increment
  - Deduce Unreachable
  - Handle Weakrefs Callbacks
  - Finalize Garbage
  - Handle Resurrected
  - Clear Weakrefs
  - Delete Garbage

PERFETTO

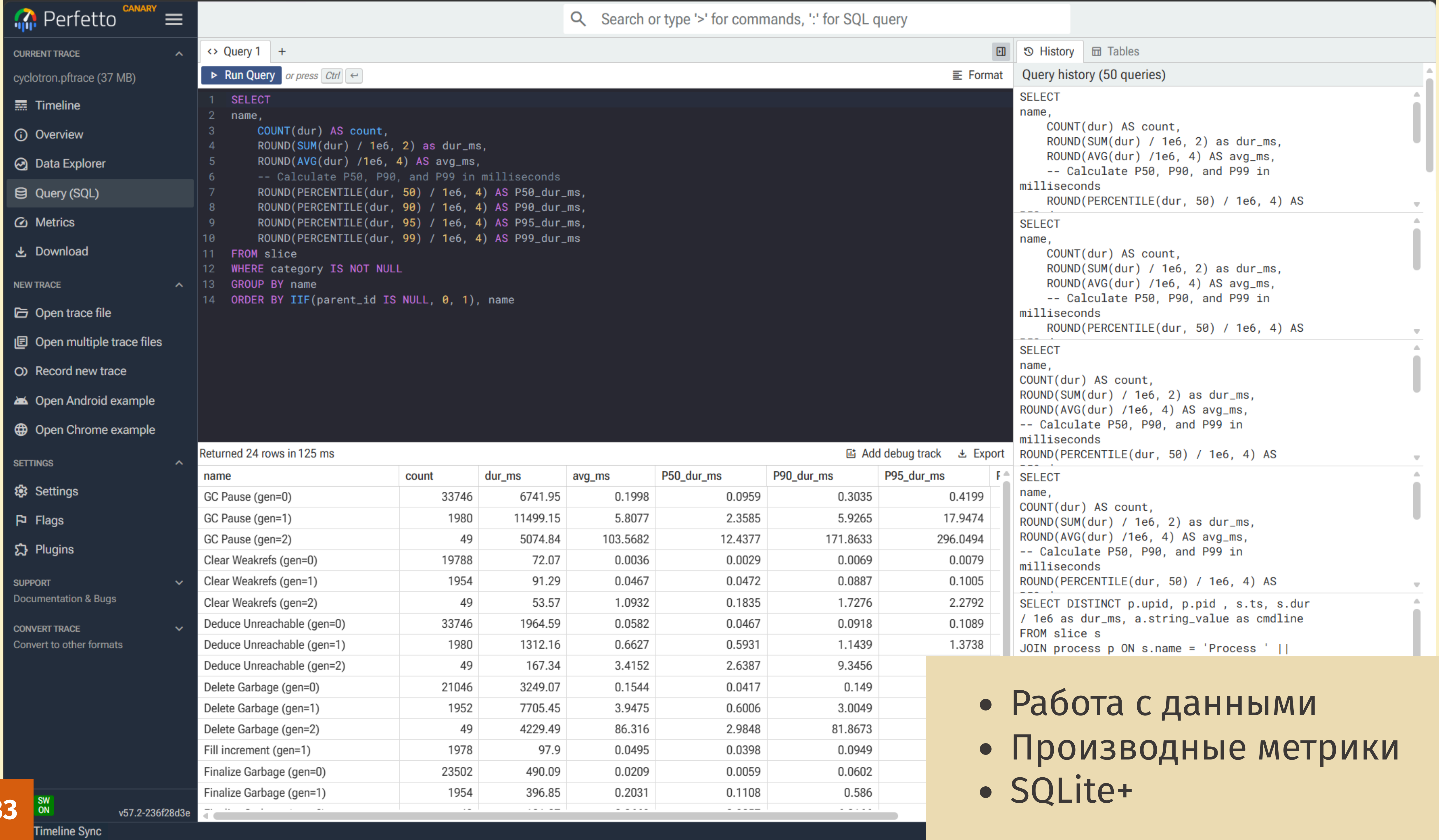
- Проблема
- Инструменты
- Новое API
- gcmon
- **Perfetto**
- Бенчмарки
- Backports



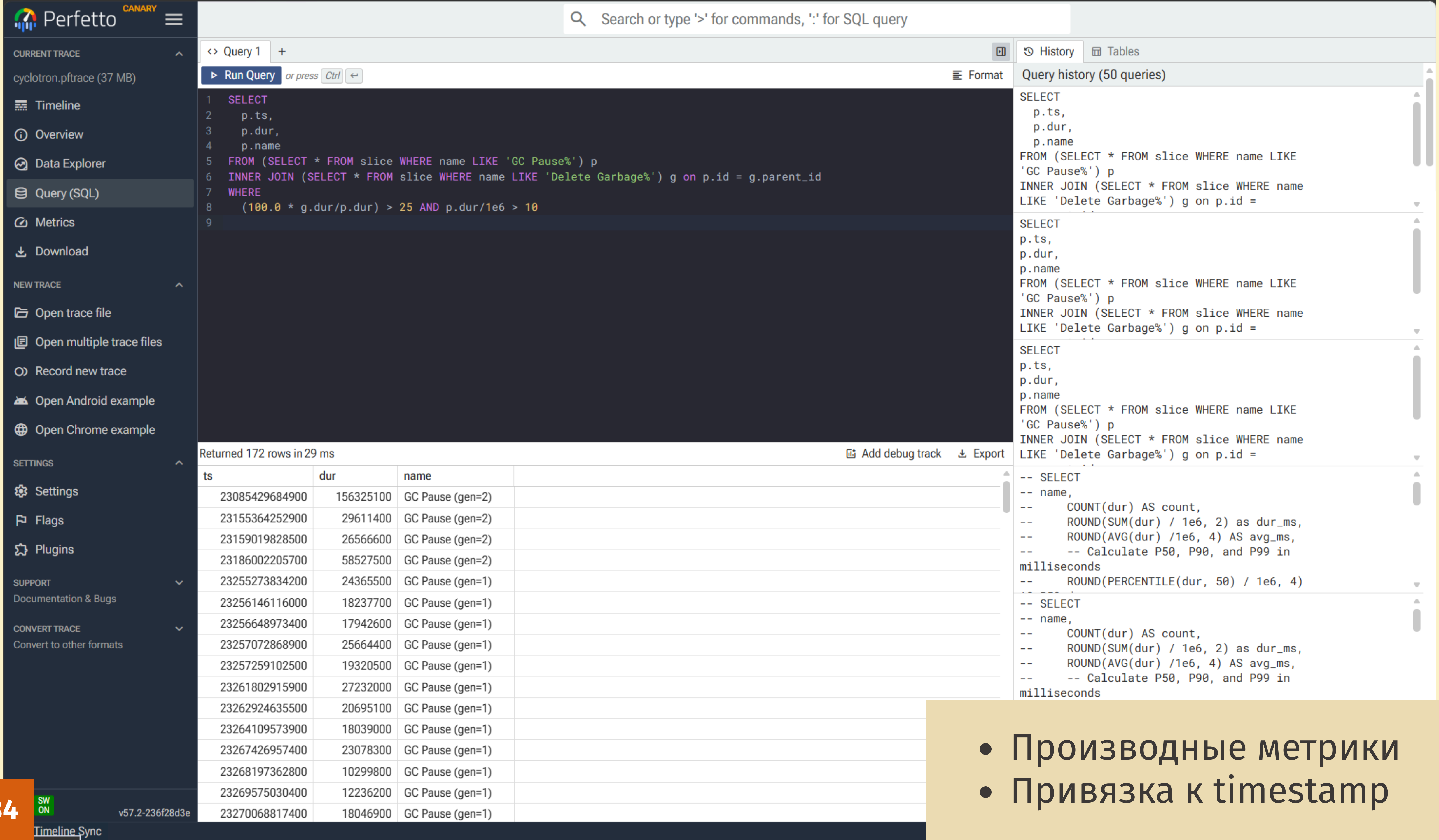
- События/Слайсы
- Метрики
- Треки
- Атрибуты
- Perfetto SQL



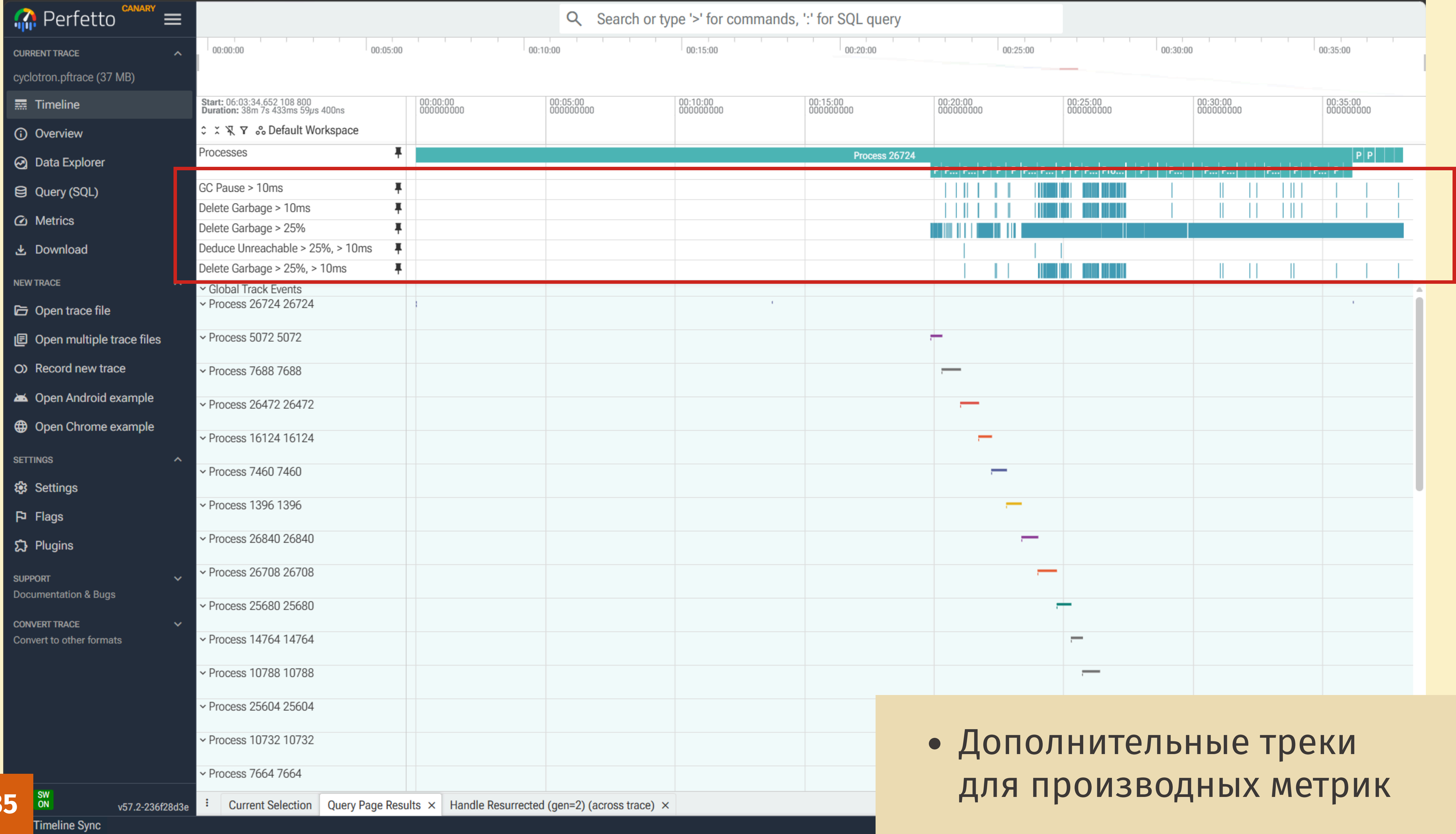
- Мини-карта процессов
- Атрибуты
- Фазы сборки мусора



- Работа с данными
- Производные метрики
- SQLite+



- Производные метрики
- Привязка к timestamp



- Дополнительные треки для производных метрик

РЕЗЮМЕ

- `gcmon` — пример использования нового API
- Позволяет получить дополнительную информацию
- Использует Perfetto для хранения и визуализации данных
- Perfetto можно использовать для своих целей

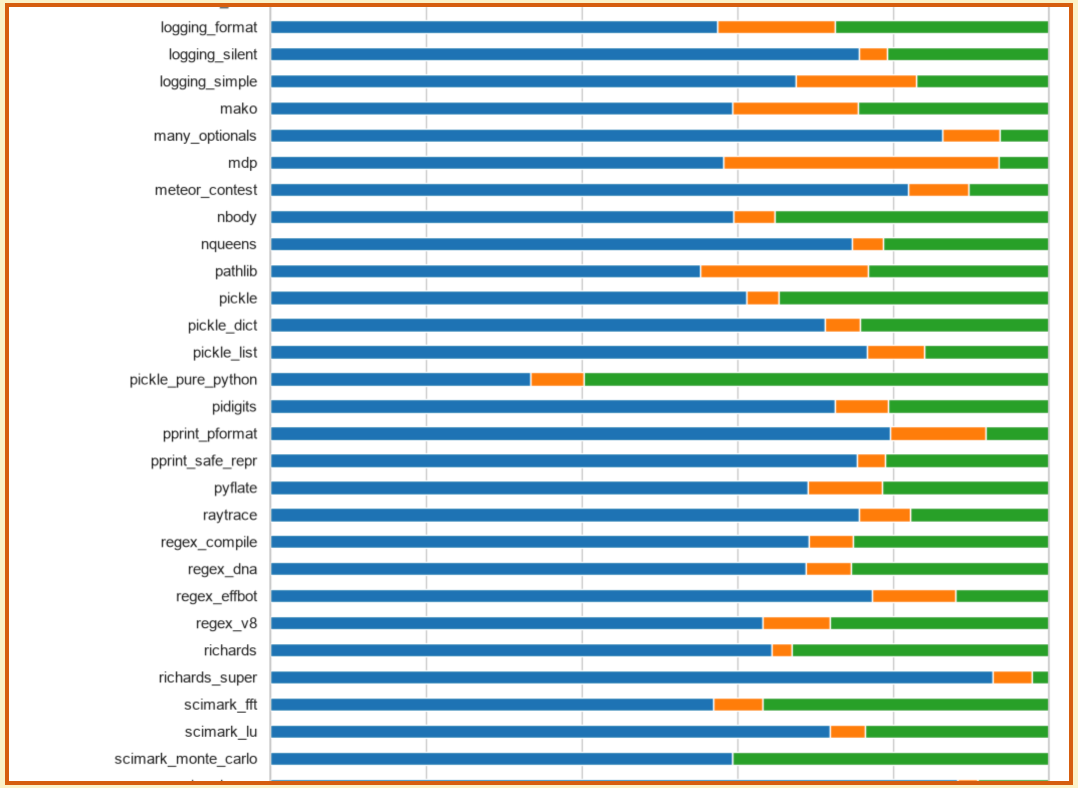
БЕНЧМАРКИ

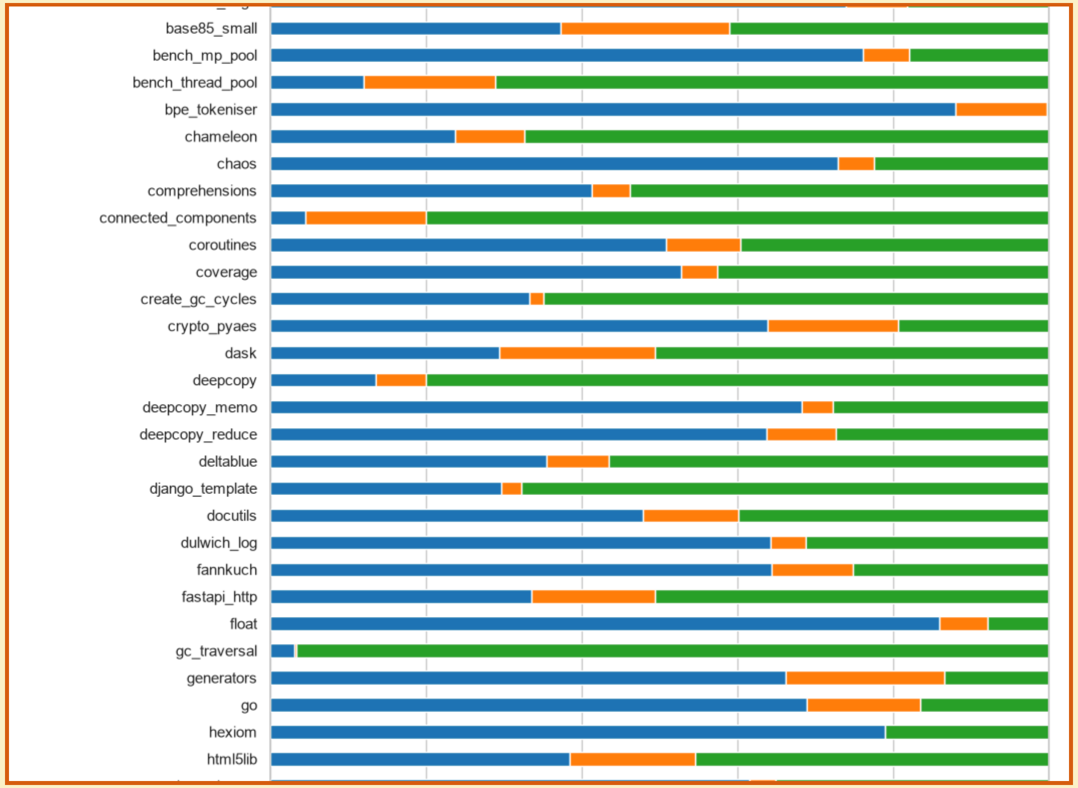
- Проблема
- Инструменты
- Новое API
- gcmon
- Perfetto
- **Бенчмарки**
- Backports

БЕНЧМАРКИ

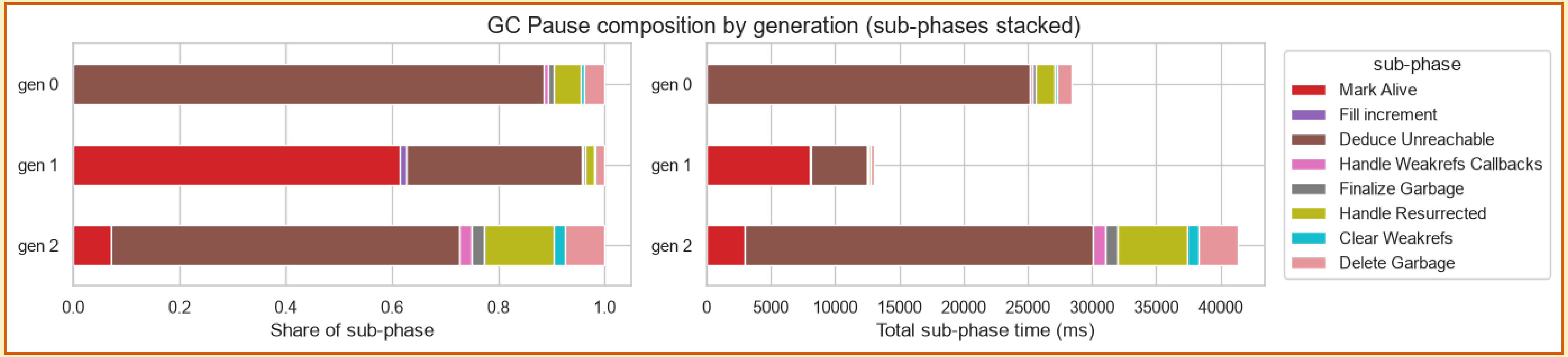
- Почему бенчмарки?
- Pyperformance
- Cyclotron
- Any other

КАКОЕ ПОКОЛЕНИЕ ПРЕОБЛАДАЕТ?

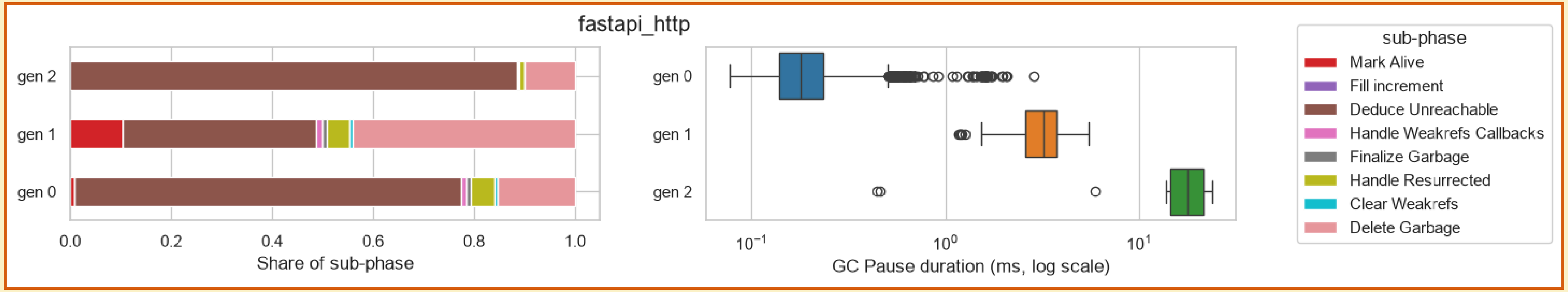




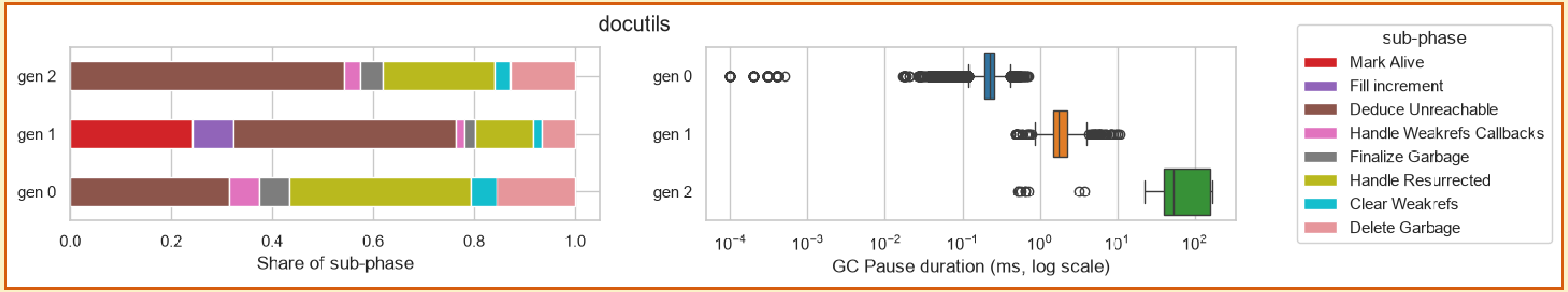
ТИПИЧНОЕ РАСПРЕДЕЛЕНИЕ ФАЗ



МНОГО ОБЪЕКТОВ С «ТЯЖЁЛЫМИ» ДАННЫМИ



ФИНАЛИЗАТОРЫ



РЕЗЮМЕ

- `gcmon` / новое API
    - можно использовать для профилирования продуктивных систем
    - можно использовать для анализа работы сборщика мусора
- Можно получить информацию о возможном тюнинге продуктивных систем

BACKPORTS

Проблема

Инструменты

Новое API

gcmon

Perfetto

Бенчмарки

Backports

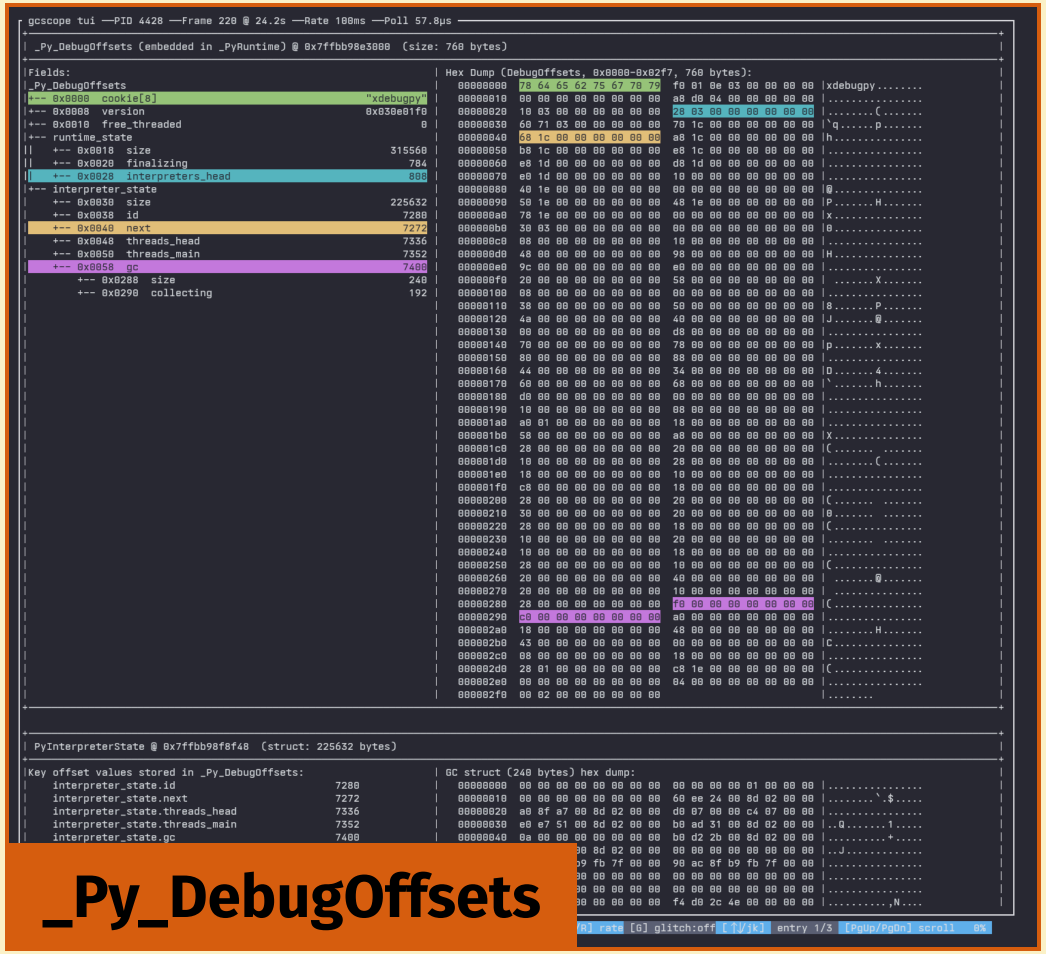
3.13-3.14

```c
typedef struct _Py_DebugOffsets {
    char cookie[8] _Py_NONSTRING;
    uint64_t version;
    uint64_t free_threaded;
    // Runtime state offset;
    struct _runtime_state {
        uint64_t size;
        uint64_t finalizing;
        uint64_t interpreters_head;
    } runtime_state;
    // Interpreter state offset;
    struct _interpreter_state {
        uint64_t size;
        uint64_t id;
        uint64_t next;
        uint64_t gc;
        // ...
    } interpreter_state;
    struct _gc {
        uint64_t size;
        uint64_t collecting;
    } gc;
} _Py_DebugOffsets;
```

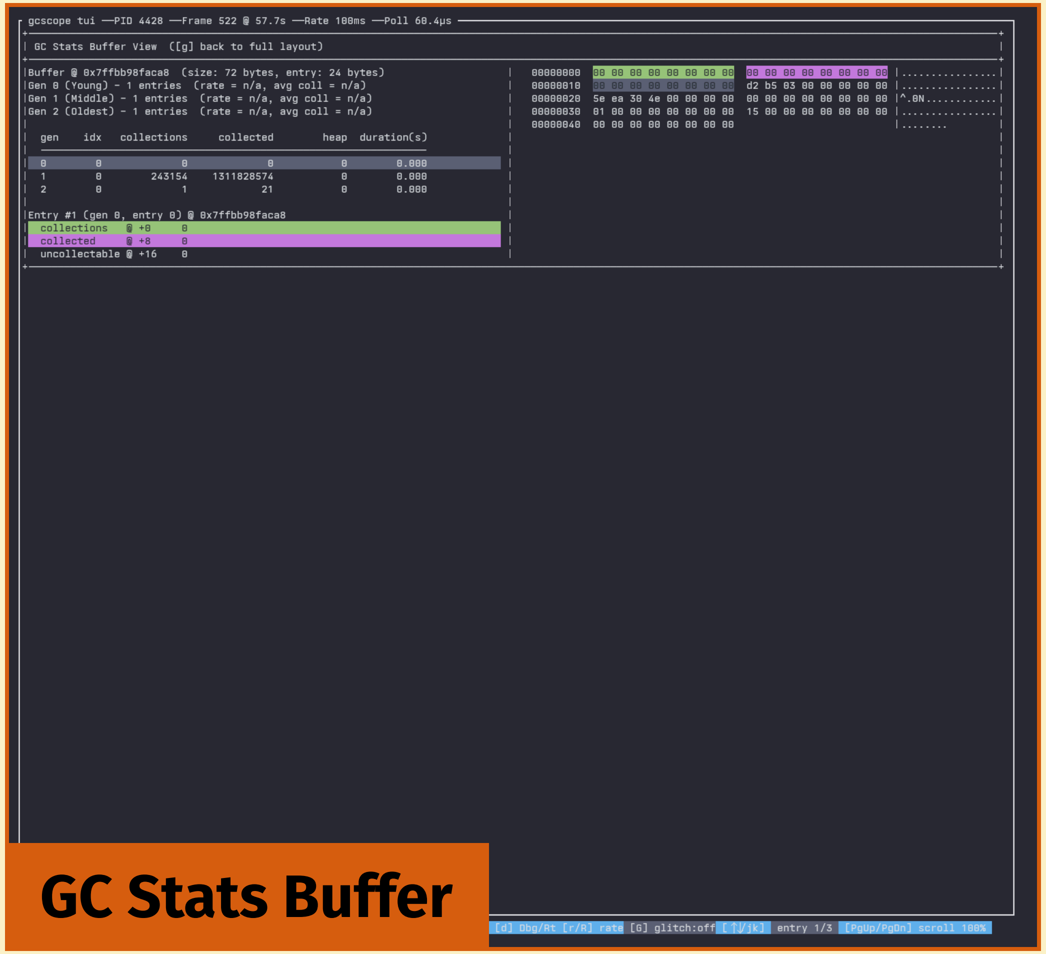
```c
/* Running stats per generation */
struct gc_generation_stats {
    Py_ssize_t collections;
    Py_ssize_t collected;
    Py_ssize_t uncollectable;
};
```

```c
struct _gc_runtime_state {
    PyObject *trash_delete_later;
    int trash_delete_nesting;
    /* Is automatic collection enabled? */
    int enabled;
    int debug;
    /* linked lists of container objects */
    struct gc_generation generations[NUM_GENERATIONS];
    PyGC_Head *generation0;
    /* a permanent generation which won't be collected */
    struct gc_generation permanent_generation;
    struct gc_generation_stats generation_stats[NUM_GENERATIONS];
    /* true if we are currently running the collector */
    int collecting;
    /* list of uncollectable objects */
    PyObject *garbage;
    /* a list of callbacks to be invoked when collection is performed */
    PyObject *callbacks;
};
```

3.13-3.14

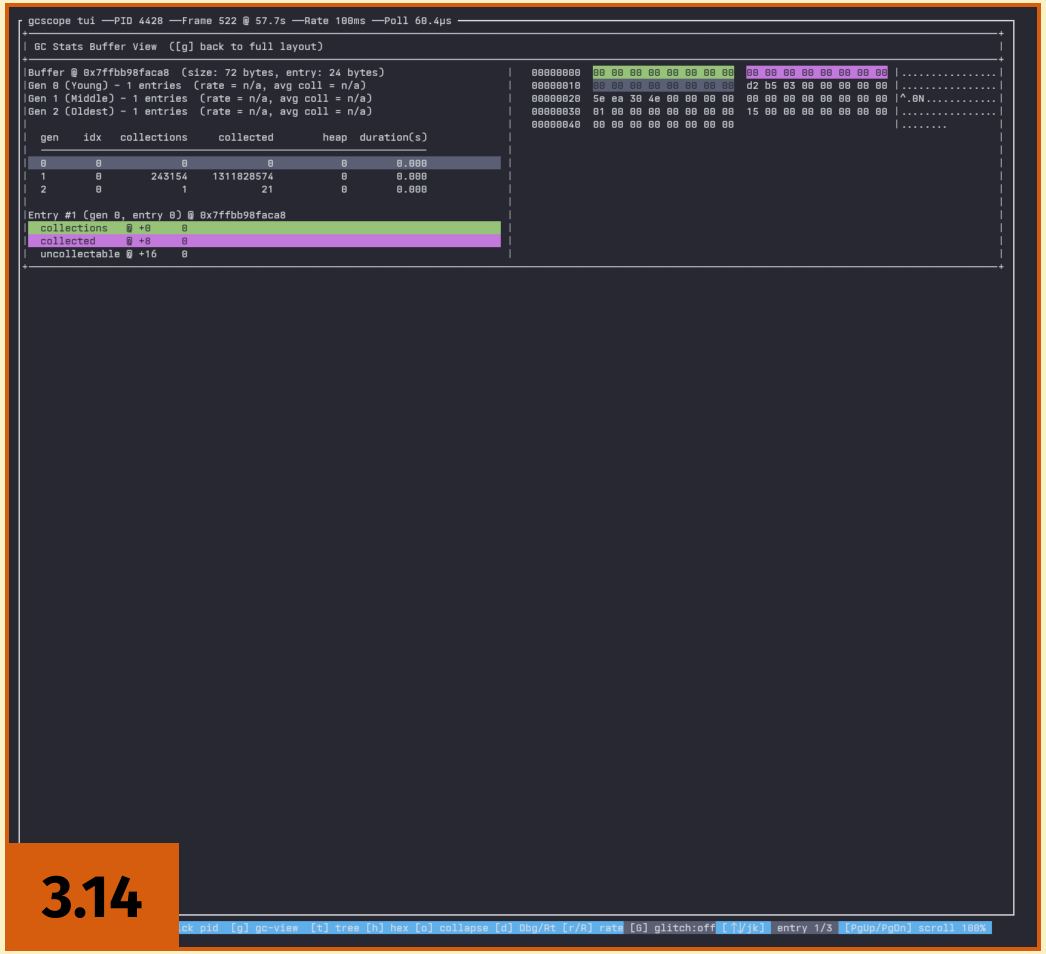


_Py_DebugOffsets

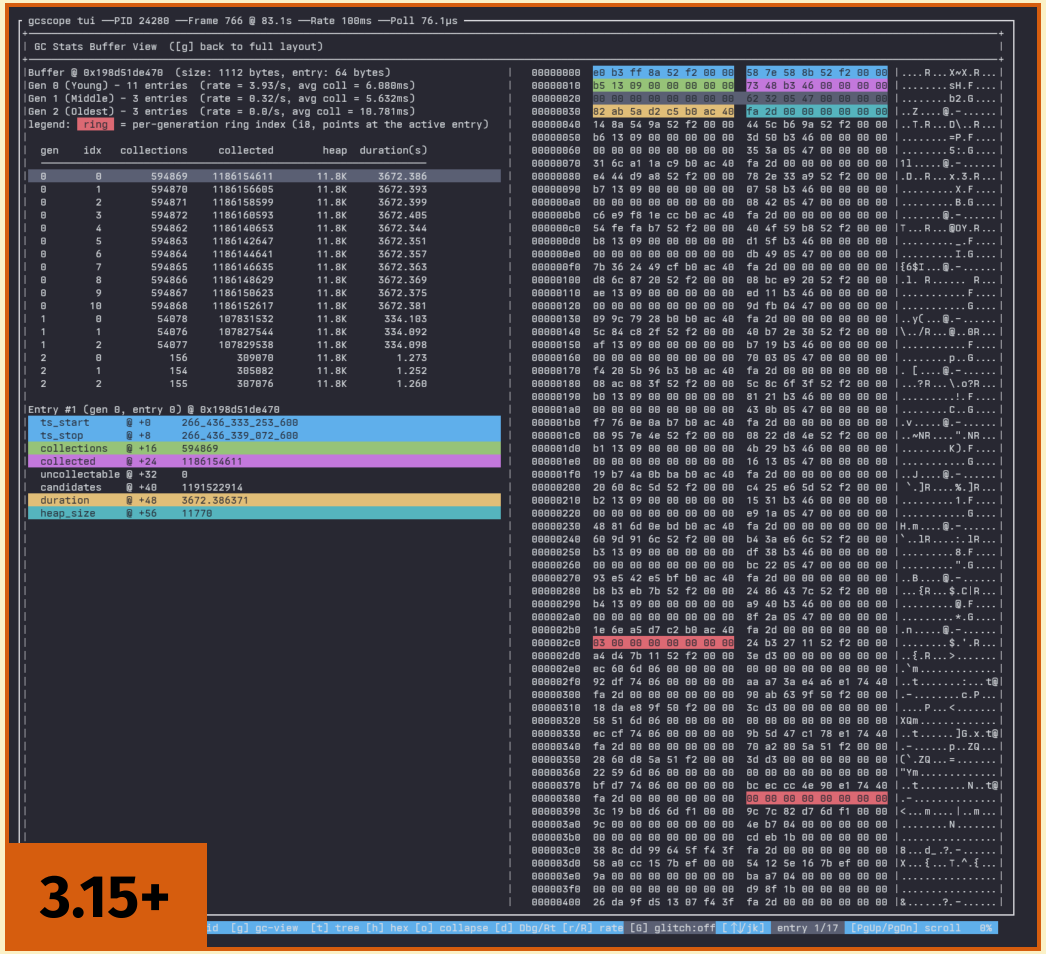


GC Stats Buffer

3.13-3.14



3.14



3.15+

3.8, 3.9-3.10, 3.11-3.12

- А если маркера нет, ничего нет, что делать?
- Насколько это стабильно?
- Какие факторы могут повлиять?
- Как это валидировать?

CALL TO ACTION

- Какие метрики вам интересны и/или необходимы?
- В каких сценариях вы бы использовали эти возможности?
- Какие форматы и интеграции вам интересны?
- Какое ваше окружение?
- Какие алерты вы видите?

Куда писать:

- gcmon/issues
- discuss.python.org
- CPython/issues

СПАСИБО ЗА ВНИМАНИЕ!

email: sergey.miryanov@gmail.com